

ARL Rover Safety Monitor Debugging Report

Name: John Maged Naiem

ID:2501293

Project: ARL Rover Safety debugging challenge

1. How corrected System Processes input to braking decision

1. Input detection data are processed for being invalid by isFinite function
2. Heading direction is converted to radians to calculate sine and cosine of the angle
3. The program iterates over the detections recorded to calculate the range and check that confidence and range are in the valid range
4. Detections which are outside the valid range of confidence or range or have invalid data from isFinite functions are skipped
5. Valid detections (obstacles) data are recorded and stored including their global X and Y coordinates, range, forward and left distances and their identity
6. Nearest obstacle is found and stored
7. Stopping distance is calculated by taking considering the reaction time distance and distance covered during braking
8. Emergency Brake is decided to be applied or not depending on the 3 features of (forward distance to obstacle, forward distance compared to stopping distance, whether it is in the lane)

2. Bugs and Corrections

2.1 Missing CMake Source Inclusion

Symptom: the compiler didn't run and gave an error of undefined functions

Root Cause: the safety_monitor.cpp was not included in the add_executable function in CmakeList

Fix: the the safety_monitor.cpp is added to the function

Reasoning & Programming Concept: Cmake couldn't find the functions in the cpp file as they are not compiled as they are not in the CmakeList causing these functions to be undefined

2.2 Missing Library Header (<optional>)

Symptom: the compiler showed an error of 'does not name a type' on the optional part causing the program not to run

Root Cause: the optional library was not included in the code

Fix: we added the optional library in code at the beginning

Reasoning & Programming Concept: the program can't work with undefined libraries they must be defined and loaded to the code from the beginning before using them

2.3 Degree to Radian Conversion Error

Symptom: the obstacle location was completely wrong

Root Cause: we used sine and cosine functions on angles in degrees not rad

Fix: we converted the degree angles to rad first by $*(\pi/180)$

Reasoning & Programming Concept: trig function must have the angles entered to them in rad so when entering degree instead the output is incorrect

2.4 Resolving incorrect sign in worldx

Symptom: the obstacle X coordinate location was wrong

Root Cause: the sign in $\sin * \text{left}$ was positive

Fix: changed the sign to negative

Reasoning & Programming Concept: when resolving the left component is $\cos(\theta + 90) = -\sin(\theta)$ so we must subtract not add

2.5 Kinematic Unit Conversion (kph to m/s)

Symptom: stopping distance was incorrect

Root Cause: didn't convert from Km/hr to m/s

Fix: divided by 3.6 to do the conversion

Reasoning & Programming Concept: reaction time was in seconds and max deceleration is in ms^2/s so must convert the speed to m/s

2.6 Index error in for loop

Symptom: the for loop didn't iterate over all the detections as valid detection remain low

Root Cause: the index+1 in the condition of the loop will cause the last detection to be processed

Fix: removed +1 made it index<detection.size()

Reasoning & Programming Concept: for example if I have only one detection the condition will be 1<1 causing the loop to not even begin with only one detection to be processed

2.7 Confidence incorrect range

Symptom: the valid detection was still low

Root Cause: the code checked that confidence is below minimum and above 0 and made this the valid condition

Fix: checked that confidence is in the true valid range >minimum and <1.0

Reasoning & Programming Concept: the code filled the logic made the invalid values valid and vice versa as the range it checked for was not the required one

2.8 Found the further not the nearest

Symptom: the location of the nearest obstacle was way too high

Root Cause: the logic was finding the largest range not smallest by using > operator

Fix: changed the > to < to find the lowest range

Reasoning & Programming Concept: using the > operator will find the largest range so the further obstacle and we need the nearest so must find the minimum not max

2.9 Emergency Brake Logic

Symptom: the emergency brake always was clear after correcting every thing

Root Cause: there was no code in the function to test the conditions and apply the required logic to decide whether clear or not

Fix: added the needed loop and if statement to apply the required rules

Reasoning & Programming Concept: with no code to check the logic the function will always return false causing clear brake no matter what so a control flow is essential here

2. Build and Execution Verification

```
[ 66%] Building CXX object CMakeFiles/rover_safety_monitor.dir/safety_mon
[100%] Linking CXX executable rover_safety_monitor.exe
[100%] Built target rover_safety_monitor
PS C:\Users\Admin\debugging assignment> .\build\rover_safety_monitor.exe
ARL Rover Safety Monitor
Valid detections: 3
Nearest obstacle: cone
World position: (96.0, 62.0)
Stopping distance: 20.0 m
Emergency brake: ENGAGE
PS C:\Users\Admin\debugging assignment> █
```

3. References & Resources

- C++ Standard Reference (cppreference.com)
- CMake Documentation
- used Gemini AI to understand CMake and how to set it up in VS code as I faced many errors when setting it and used to make this empty template that I filled